

GhostPredict: compressão lossless com prior embutido no codec para micro-payloads de IoT

Matheus Araújo

Projeto de pesquisa independente

araujomatheuscarv@gmail.com

Abstract.

General purpose compressors such as gzip, zstd and brotli enlarge very short messages, because header and framing overhead exceeds any saving. This work presents GhostPredict, a lossless compressor specialized in the micro-payloads of tens of bytes that dominate IoT telemetry. The method combines a header free format, an order-2 context model over nibbles with integer arithmetic coding, and a domain prior embedded in the codec rather than in the file, which both warms the statistical model and acts as an LZ dictionary. Across eleven real datasets the primed variant produces the smallest files among all baselines, including short string specialists and trained zstd dictionaries; once the dictionary framing is removed to isolate the model, the advantage over the lean zstd dictionary is about 1.8 times. The file stays dictionary free, and the method never expands incompressible input beyond a negligible margin. We also quantify the dependence of the gains on a domain matched prior.

Resumo.

Compressores de propósito geral como gzip, zstd e brotli aumentam o tamanho de mensagens muito curtas, pois o custo de cabeçalho e enquadramento supera qualquer economia. Este trabalho apresenta o GhostPredict, um compressor lossless especializado nos micro-payloads de dezenas de bytes da telemetria de IoT. O método combina um formato sem cabeçalho, um modelo de contexto de ordem 2 sobre nibbles com codificação aritmética inteira, e um prior de domínio embutido no codec, e não no arquivo, que aquece o modelo e serve de dicionário. Em onze conjuntos de dados reais, a variante com prior produz o menor arquivo entre todos os concorrentes, incluindo especialistas de string curta e o dicionário treinado do zstd; isolando o modelo, ao remover o enquadramento do dicionário, a vantagem sobre o zstd enxuto é de cerca de 1,8 vezes. O arquivo permanece livre de dicionário e o método nunca expande entradas incompressíveis além de margem desprezível. Também quantificamos a dependência dos ganhos em relação a um prior casado com o domínio.

1. Introdução

A expansão da Internet das Coisas multiplicou um tipo de tráfego que os compressores clássicos atendem mal: mensagens muito curtas, frequentemente de trinta a uma centena de bytes, emitidas por sensores, rastreadores e dispositivos de telemetria. Protocolos como MQTT e CoAP, e enlaces de baixa potência como LoRaWAN e NB-IoT, transportam esse tipo de payload, no qual cada byte transmitido tem custo direto em energia, em tempo de antena e em limites regulatórios de ciclo de trabalho.

Nesse regime de tamanho, os compressores de propósito geral tendem a aumentar o dado em vez de reduzi-lo. Formatos como o DEFLATE [Deutsch 1996], o Zstandard [Collet and Kuchera 2021] e o Brotli [Alakuijala and Szabadka 2016] carregam cabeçalhos, somas de verificação e estruturas de bloco cujo custo fixo domina quando a entrada tem poucas dezenas de bytes. O resultado é que uma mensagem de trinta bytes costuma sair maior do que entrou.

Este trabalho investiga se um compressor desenhado para esse nicho pode reverter o quadro. A proposta, chamada GhostPredict, parte de três decisões de projeto: um formato sem cabeçalho, em que o arquivo é apenas o fluxo aritmético; um prior de domínio embutido no próprio codec, e não no arquivo, que aquece o modelo estatístico e fornece um dicionário inicial, eliminando o custo de partida a frio; e uma garantia de que nenhuma entrada será expandida além de margem mínima. As contribuições são o algoritmo e o formato, o mecanismo de prior embutido, a garantia de não expansão, e uma avaliação em onze conjuntos de dados reais que isola o ganho do modelo do ganho de formato e quantifica a dependência do prior em relação ao domínio. O código e os scripts de reprodução estão disponíveis publicamente em <https://github.com/matheusaraujoc/GPA>.

2. Trabalhos relacionados

A compressão lossless de propósito geral apoia-se na substituição de cadeias repetidas por referências [Ziv and Lempel 1977] e na modelagem estatística seguida de codificação de entropia, cujos marcos são o código de Huffman [1952], a predição por casamento parcial de Cleary and Witten [1984] e a codificação aritmética de Witten, Neal and Cleary [1987]. O GhostPredict combina as duas famílias.

Para mensagens curtas existem compressores especializados com codebook embutido, como o Unishox2 [Ramanathan 2021] e o SMAZ, usados em dispositivos restritos. Esses métodos vencem a partida a frio por já carregarem conhecimento sobre a língua, mas o conhecimento é estático e não se adapta ao domínio. A ideia de compartilhar um prior entre os dois lados, sem inseri-lo no arquivo, tem precedente maduro: o zstd treina dicionários a partir de amostras, o concorrente direto da nossa proposta; e, em genômica, a compressão baseada em referência [Fritz et al. 2011] é o paradigma dominante. O GhostPredict transpõe esse princípio para o extremo dos micro-payloads, onde o custo fixo dos formatos gerais é proporcionalmente maior.

3. O algoritmo GhostPredict

O núcleo é um pipeline determinístico: a entrada passa por um estágio LZ77 de janela deslizante, que substitui repetições por pares de distância e comprimento; o fluxo de símbolos resultante, com nibbles literais e códigos de distância e comprimento, alimenta um modelo de contexto de ordem 2 sobre nibbles; e as probabilidades guiam um codificador aritmético inteiro de 32 bits. Por usar aritmética inteira, a codificação é exatamente reproduzível, garantindo o casamento bit a bit entre compressão e descompressão.

O arquivo é apenas o fluxo aritmético, sem cabeçalho. Para impedir expansão de entradas incompressíveis, a primeira decisão codificada é uma escolha binária enviesada entre o modo normal e um modo de armazenamento direto, na proporção de um para sessenta e quatro. O custo dessa flag é cerca de 0,02 bit no caso compressível e seis bits no armazenado, onde os bytes são codificados por um modelo plano, limitando a expansão máxima a cerca de um por mil mais alguns bytes.

A característica central é o modo com prior. Antes de codificar a mensagem, compressor e descompressor percorrem um corpus de domínio, o primer, atualizando o modelo de forma idêntica, sem emitir bits, e o utilizam como dicionário do LZ77. O primer reside no codec e é distribuído uma vez junto ao firmware; não acompanha cada mensagem. Cada arquivo permanece livre de dicionário, e o ganho do prior se soma à ausência de custo fixo.

4. Metodologia experimental

A avaliação usa onze conjuntos de dados reais, listados na Tabela 1, todos como fluxos de registros curtos com um registro por linha. Para cada domínio, os registros são embaralhados com semente fixa e divididos em treino e teste disjuntos. O conjunto de treino constrói tanto o primer do GhostPredict, formado pelos últimos 32 KB do corpus de treino, quanto o dicionário do zstd, treinado pelas mesmas amostras, de modo que ambos recebem a mesma informação prévia. O conjunto de teste, com mil mensagens não vistas, é comprimido mensagem a mensagem. Medir mensagens individuais, e não arquivos inteiros, é essencial: medir arquivos diluiria o custo de partida a frio e esconderia o efeito que o prior corrige. A exceção de tamanho são os quatro conjuntos de log, na versão amostrada de duas mil linhas do Loghub, em que teste e treino têm mil mensagens cada.

Tabela 1. Conjuntos de dados reais utilizados na avaliação.

Domínio	Fonte	Registro típico
MQTT / IoT	MQTTEEB-D [Aqachtoul et al. 2025]	~76 B
AIS / navios	MarineCadastre [MarineCadastre 2025]	~100 B
Sensores	Intel Lab Data [Bodik et al. 2004]	~64 B
Energia	UCI Household Power [Hebrail and Berard 2012]	~62 B

GPS	GeoLife [Zheng et al. 2009]	~65 B
Logs (4 tipos)	Loghub [Zhu et al. 2023]	84 a 300 B
SMS	SMS Spam Collection [Almeida et al. 2011]	~88 B
Tweets	Sentiment140 [Go et al. 2009]	~148 B

Comparam-se dez configurações: o GhostPredict sem prior e com prior; os especialistas de string curta Unishox2 e SMAZ; o gzip nível 9; o zstd nível 19, com e sem dicionário treinado; uma variante enxuta do dicionário do zstd, sem soma de verificação e sem identificador de dicionário, que remove o enquadramento removível para isolar a contribuição do modelo; o brotli nível 11; e o xz nível 9. As versões foram zstd 1.5.7, xz 5.6.3, gzip 1.13, brotli 1.1.0 e Unishox2 da distribuição oficial. As métricas são o tamanho médio, a mediana e o percentil 95 por mensagem, a fração de mensagens expandidas, a fração de vitórias, a vazão e o uso de memória. O procedimento é reproduzível a partir do repositório público (<https://github.com/matheusaraujoc/GPA>), com sementes fixas; o commit exato e um identificador de versão devem acompanhar a versão final.

5. Resultados

A Tabela 2 traz o tamanho médio do comprimido, em bytes, por domínio e configuração, incluindo a variante enxuta do dicionário do zstd e os especialistas de string curta. Valores menores são melhores.

Tabela 2. Tamanho médio do comprimido por mensagem (bytes). Menor é melhor.

Domínio	orig	GPA frio	GPA prior	unishox2	smaz	zstd dict	zstd lean	brotli
MQTT (IoT)	75,33	47,71	17,66	39,85	97,1	41,56	33,56	52
AIS (navios)	101,02	82,89	44,1	66,15	140,57	68,53	60,53	80,28
Sensores (Intel Lab)	63,48	48,74	26,08	39,21	91,02	51,87	43,87	47,63
Medidor (energia)	62,86	40,83	17,42	36,7	81,97	39,12	31,12	39,13
GPS (GeoLife)	64,95	51,3	23,93	37,39	88,26	48,62	40,62	47,05
Log Apache	83,51	75,14	10,1	58,23	81,75	31,96	23,96	78,9
Log Linux	106,89	90,29	14,9	71,57	105,55	36,04	28,04	96,38
Log OpenSSH	109,38	95,78	14,28	78,67	110,9	36,41	28,41	95,65
Log HDFS	141,76	114,49	32,12	94,78	151,92	53,29	45,29	117,53
SMS	80,63	66,63	38,99	52,72	57,48	63,91	55,91	67,01
Tweets	71,48	60,41	35,57	46,21	47,62	60,16	52,16	60,69

Em todos os domínios o GhostPredict com prior produz o menor arquivo médio. Sobre o dicionário treinado do zstd, o fator de redução varia de 1,55 a 3,16 vezes, média 2,12. Esse número, porém, inclui o enquadramento do zstd. Contra a variante enxuta, que remove a soma de verificação e o identificador de dicionário, o fator cai para 1,37 a

2,37 vezes, média 1,73, valor que reflete apenas o modelo. Sobre o Unishox2, especialista sem prior, a média é 2,78 vezes.

A diferença entre as duas comparações é o custo fixo de enquadramento. Medido sobre uma entrada vazia, o dicionário do zstd ocupa 17 bytes na forma padrão e 9 bytes na forma enxuta, custo que o GhostPredict não paga por não ter cabeçalho. A Tabela 3 decompõe, em bytes médios por mensagem, a vantagem do GhostPredict com prior sobre o dicionário do zstd em duas parcelas: a moldura removível, que é a soma de verificação e o identificador de dicionário eliminados pela variante enxuta, e o restante, que combina o modelo aquecido e o cabeçalho irreduzível do zstd que o formato sem cabeçalho também evita.

Tabela 3. Decomposição da vantagem do GPA com prior sobre o zstd-dict (bytes/mensagem).

Domínio	Vantagem total	Moldura removível	Modelo + formato
MQTT (IoT)	23,9	8,0	15,9
AIS (navios)	24,4	8,0	16,4
Sensores (Intel Lab)	25,8	8,0	17,8
Medidor (energia)	21,7	8,0	13,7
GPS (GeoLife)	24,7	8,0	16,7
Log Apache	21,9	8,0	13,9
Log Linux	21,1	8,0	13,1
Log OpenSSH	22,1	8,0	14,1
Log HDFS	21,2	8,0	13,2
SMS	24,9	8,0	16,9
Tweets	24,6	8,0	16,6

A Tabela 4 mostra que a vantagem não se deve a poucos casos extremos: para o GhostPredict com prior, a mediana e o percentil 95 do tamanho por mensagem ficam próximos da média em todos os domínios.

Tabela 4. GhostPredict com prior: média, mediana e percentil 95 do tamanho (bytes).

Domínio	Média	Mediana	P95
MQTT (IoT)	17,66	17	22
AIS (navios)	44,1	45	55
Sensores (Intel Lab)	26,08	26	30
Medidor (energia)	17,42	17	21
GPS (GeoLife)	23,93	24	31
Log Apache	10,1	10	13
Log Linux	14,9	13	30
Log OpenSSH	14,28	13	23
Log HDFS	32,12	30	40
SMS	38,99	28	89
Tweets	35,57	33	66

A Tabela 5 mostra a fração de mensagens que cada ferramenta expande e a fração em que o GhostPredict com prior produz o menor resultado. Ela evidencia o problema que motiva o trabalho: no regime de poucas dezenas de bytes, gzip e xz frequentemente aumentam o dado.

Tabela 5. Mensagens expandidas (Exp.) e vitórias (Venc.), em porcentagem.

Domínio	GPA prior	gzip Exp.	xz Exp.	smaz Exp.	unishox2 Exp.
---------	-----------	-----------	---------	-----------	---------------

	Venc.				
MQTT (IoT)	100	54,4	100	100	0
AIS (navios)	98,9	88,1	100	100	0
Sensores (Intel Lab)	100	100	100	100	0
Medidor (energia)	100	97,6	100	100	0
GPS (GeoLife)	100	93,1	100	100	0
Log Apache	100	100	100	17,2	0
Log Linux	96,1	88,9	100	47,5	0
Log OpenSSH	99,2	79,7	100	61,9	0
Log HDFS	99,8	51	99,8	97,8	0
SMS	93,9	77,5	99,2	3,9	1,1
Tweets	95,3	81,6	100	2,2	0

A Tabela 6 quantifica a dependência do prior em relação ao domínio. Cada célula é o tamanho médio do comprimido quando se usa o primer do domínio da coluna para comprimir as mensagens de teste do domínio da linha. A diagonal, com primer casado, é sempre a menor; fora dela, o tamanho cresce de forma acentuada, podendo aproximar-se do tamanho original.

Tabela 6. Tamanho médio (bytes) com primer cruzado: linha é o teste, coluna é o primer.

teste \ primer	MQTT	AIS	Sensores	GPS	Log
MQTT	18,18	46,33	56,75	46,9	52,41
AIS	91	43,21	92,16	78,25	95,91
Sensores	54,09	46,73	26,14	47,23	50,15
GPS	49,16	42,52	51,55	23,71	56,24
Log	99,14	97,29	102,03	103,2	9,82

Quanto a recursos, é preciso separar duas medidas distintas. A Tabela 7 traz a vazão de compressão e o pico de memória residente do processo no computador de teste, sobre lotes concatenados. Esse pico de processo inclui o binário e o sistema operacional e não representa o consumo do algoritmo embarcado. O número relevante para microcontroladores é o conjunto de trabalho do motor, medido por um alocador instrumentado e independente do tamanho da mensagem no perfil micro: cerca de 19 KB para o decodificador e cerca de 51 KB para o compressor. A esse custo soma-se o primer, que ocupa de 16 a 32 KB na memória de programa do dispositivo, pois reside no codec e não no arquivo.

Tabela 7. Vazão média e pico de memória do processo no host (não é o footprint embarcado).

Configuração	Vazão (MB/s)	Mem. proc. (MB)
GPA frio	7,0	7,1
GPA prior	7,2	9,3
gzip-9	9,8	6,6
zstd-19	10,0	51,9
brotli-11	3,1	20,5
xz-9	4,3	42,8

A Tabela 8 reúne, em um único lugar, os custos do prior, hoje dispersos pelo texto. Ela torna o trade-off imediatamente visível: um custo fixo de memória de programa e de RAM, em troca da compressão, condicionado à estabilidade do domínio.

Tabela 8. Custos do prior (resumo do trade-off).

Item	Valor
Primer (memória de programa)	16 a 32 KB
Decodificador (RAM de trabalho)	cerca de 19 KB
Compressor (RAM de trabalho)	cerca de 51 KB
Degradação com domínio cruzado	até cerca de 10 vezes (Tabela 6)

A título de ilustração, uma mensagem MQTT real de 69 bytes, "1725866030.472465,0x0018,...,2,dos", é reduzida pelo GhostPredict com prior a 16 bytes. As demais ferramentas, sem um prior casado, ficam bem acima: o Unishox2 produz 35 bytes, o zstd com dicionário 43 bytes, sua variante enxuta 35 bytes, e o GhostPredict sem prior 42 bytes.

Um exemplo de log esclarece por que os logs comprimem tanto. A linha do Apache "[Sun Dec 04 06:06:11 2005] [notice] workerEnv.init() ok /etc/httpd/conf/workers2.properties", de 91 bytes, é reduzida pelo GhostPredict com prior a 10 bytes, ante 30 bytes do zstd com dicionário, 63 do Unishox2 e 82 do GhostPredict sem prior. O motivo é que essa linha é um molde quase fixo, repetido milhares de vezes no fluxo de log; o primer já o conhece, de modo que apenas a parte variável, sobretudo o instante de tempo, precisa ser codificada. A agressividade dos resultados de log decorre, portanto, da natureza altamente repetitiva desse tráfego, e não de um artefato de medição.

Os dois exemplos, o do MQTT e o do log, resumem em uma única mensagem o efeito do prior de domínio e a razão dos ganhos observados nas tabelas anteriores.

6. Discussão

Os resultados sustentam a tese e delimitam seu alcance. No nicho de micro-payloads, o prior embutido no codec entrega o menor arquivo entre todos os concorrentes. É importante separar as duas fontes desse ganho. Parte vem do formato sem cabeçalho, pois o dicionário do zstd carrega um enquadramento fixo que, em saídas de algumas dezenas de bytes, representa uma fração relevante; ao remover o enquadramento removível, na variante enxuta, a vantagem do GhostPredict cai mas permanece. A parte restante vem do modelo aquecido pelo prior, que reduz o custo de codificação das mensagens. Ambas as fontes são contribuições legítimas, e a decomposição evita atribuir ao modelo um ganho que é, em parte, de formato.

Fora do nicho o método não compete. Em dados gerais de maior porte a razão de compressão fica na classe do gzip, atrás de zstd, xz e brotli, pela janela curta e pelo modelo de baixa ordem. Em texto de linguagem natural livre, compressores de caractere permanecem competitivos. A principal ameaça à validade é a dependência entre o prior e o domínio, quantificada na Seção 5: um prior de domínio distinto degrada o resultado e pode até inflar, o que reforça a necessidade de casar o primer com a aplicação. A avaliação usa mil mensagens por domínio; uma validação industrial deveria ampliar amostras e domínios.

Estabelecido que o efeito existe e medido onde ele vem, a questão prática deixa de ser a sua realidade e passa a ser o seu balanço de engenharia. A proposta troca um custo fixo por um ganho recorrente. O custo é o primer, de 16 a 32 KB na memória de programa do dispositivo, pago uma única vez e nunca transmitido. O ganho é a redução de cerca de duas a três vezes sobre o dicionário do zstd em cada mensagem enviada, ou de cerca de 1,7 vezes quando se isola o modelo do formato. Em um cenário típico de IoT, com um dispositivo que transmite muitas mensagens curtas por enlaces onde cada byte custa energia e tempo de antena, o balanço é favorável, pois a memória de programa investida é abundante nos microcontroladores usuais, da ordem de megabytes, enquanto a economia se acumula a cada transmissão. A condição para que o trade-off compense é a estabilidade do domínio, quantificada na Seção 5: um primer descasado anula o ganho e pode inflar. O método é, portanto, indicado quando o domínio é estável e o volume de mensagens é alto, e contraindicado quando o tráfego é heterogêneo ou a memória de programa é criticamente escassa.

7. Viabilidade em hardware embarcado

O destino natural do método são dispositivos de recursos restritos, como os microcontroladores das famílias ESP32, STM32 e nRF, onde o ganho em bytes se converte em economia de energia e de tempo de antena. A arquitetura de uso é assimétrica: o dispositivo comprime a mensagem antes de transmitir, com o codificador e o primer gravados no firmware, e o gateway ou a nuvem descomprime. Como todo o pipeline usa apenas aritmética inteira e é determinístico, ele não exige unidade de ponto flutuante e roda de forma idêntica no microcontrolador e no servidor. Os requisitos de memória cabem nesses dispositivos: o conjunto de trabalho do decodificador, de cerca de 19 KB, e o do codificador no perfil micro, de cerca de 51 KB, ficam bem abaixo das centenas de quilobytes de RAM de um ESP32, e o primer, de 16 a 32 KB, reside na memória de programa, da ordem de megabytes nesses chips.

Boa parte dessas alegações pode ser validada sem o dispositivo físico, por compilação cruzada e análise estática. Ao portar o motor para o ambiente de destino, em C sobre o ESP-IDF ou em Rust no modo `no_std`, basta compilar o binário no computador e inspecionar seu mapa de memória. No ESP-IDF, o comando `idf.py size` reporta a divisão em flash, com as seções de código (`.text`) e de constantes (`.rodata`, onde reside o primer), e em RAM estática, com as seções de dados inicializados (`.data`) e não inicializados (`.bss`). Esses números validam, de forma reproduzível e sem hardware, a parte estática do footprint da Tabela 8: que o código e o primer cabem na flash e que as estruturas estáticas cabem na RAM. Um projeto que aloque o modelo em vetores estáticos, em vez de no heap, torna o footprint completo diretamente visível nessa análise. Restam para o dispositivo real, ou para um emulador como o QEMU, a latência por mensagem e o pico de heap dinâmico, medíveis com `esp_timer_get_time` e `esp_get_free_heap_size`.

O balanço energético reforça a viabilidade. Em enlaces de baixa potência, a transmissão de rádio domina o consumo, de modo que gastar alguns microssegundos de processador para reduzir o payload à metade prolonga a autonomia. Em tecnologias com limites rígidos de payload, como os doze bytes por quadro do Sigfox ou os tetos de ciclo de trabalho do LoRaWAN, a redução pode ser a diferença entre caber ou não em um quadro, ou entre respeitar ou não o limite regulatório.

Essa viabilidade foi confirmada na prática. Portamos o decodificador para C sem alocação dinâmica, com todo o estado em memória estática, e o verificamos bit a bit contra o motor de referência nos vetores de conformidade e em mensagens reais. Ao compilá-lo para o ESP32 com o ESP-IDF, o relatório de tamanho por componente do `idf.py` atribui ao decodificador 19,7 KB na seção de dados não inicializados e cerca de 1,8 KB de código em flash, no alvo xtensa real. O valor de RAM converge com as outras duas medições independentes, o alocador instrumentado em Rust e a análise do objeto no hospedeiro, ambas em torno de 19 KB, o que confirma a linha do decodificador da Tabela 8 no próprio hardware de destino. Restam para o dispositivo físico, ou para o emulador, a latência e a energia.

8. Conclusão e trabalhos futuros

Apresentamos o GhostPredict, um compressor lossless para micro-payloads de IoT que combina formato sem cabeçalho, garantia de não expansão e um prior de domínio embutido no codec. A avaliação em onze conjuntos de dados reais mostra que, no seu nicho, o método produz o menor arquivo entre todos os concorrentes, e que, isolando o modelo do formato, mantém vantagem sobre o dicionário treinado do `zstd`, preservando arquivos livres de dicionário. Os ganhos observados dependem da existência de um prior compatível com o domínio da aplicação, condição que quantificamos e que define o escopo de uso do método. Como trabalho futuro, planejamos um modelo de contexto de ordem mais alta, a inclusão de mais especialistas de string curta na comparação, e a validação do footprint do decodificador em microcontroladores reais.

Referências

- Alakuijala, J. and Szabadka, Z. (2016). Brotli Compressed Data Format. RFC 7932, IETF.
- Almeida, T. A., Gómez Hidalgo, J. M. and Yamakami, A. (2011). Contributions to the study of SMS spam filtering: new collection and results. In Proceedings of the 2011 ACM Symposium on Document Engineering (DocEng), pages 259-262.
- Aqachtoul, A., Karam, K., Elamrani, A., Najib, M., Rafalia, N. and Bakhouya, M. (2025). MQTTEEB-D: a real-world IoT cybersecurity dataset for AI-powered threat detection in MQTT networks. Data in Brief, 61:111897.

- Bodik, P., Hong, W., Guestrin, C., Madden, S., Paskin, M. and Thibaux, R. (2004). Intel Lab Data. Intel Berkeley Research Lab. <http://db.csail.mit.edu/labdata/labdata.html>.
- Cleary, J. G. and Witten, I. H. (1984). Data compression using adaptive coding and partial string matching. *IEEE Transactions on Communications*, 32(4):396-402.
- Collet, Y. and Kucherawy, M. (2021). Zstandard Compression and the 'application/zstd' Media Type. RFC 8878, IETF.
- Deutsch, P. (1996). DEFLATE Compressed Data Format Specification version 1.3. RFC 1951, IETF.
- Fritz, M. H.-Y., Leinonen, R., Cochrane, G. and Birney, E. (2011). Efficient storage of high throughput DNA sequencing data using reference-based compression. *Genome Research*, 21(5):734-740.
- Go, A., Bhayani, R. and Huang, L. (2009). Twitter sentiment classification using distant supervision. Technical report, Stanford University.
- Hébrail, G. and Bérard, A. (2012). Individual Household Electric Power Consumption Data Set. UCI Machine Learning Repository. <https://doi.org/10.24432/C58K54>.
- Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098-1101.
- MarineCadastre (2025). Vessel Traffic Data (AIS). NOAA and BOEM. <https://marinecadastre.gov/accessais/>.
- Ramanathan, A. (2021). Unishox: a hybrid encoder for short Unicode strings. Zenodo. <https://doi.org/10.5281/zenodo.5800408>.
- Witten, I. H., Neal, R. M. and Cleary, J. G. (1987). Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520-540.
- Zheng, Y., Zhang, L., Xie, X. and Ma, W.-Y. (2009). Mining interesting locations and travel sequences from GPS trajectories. In *WWW 2009*, pages 791-800.
- Ziv, J. and Lempel, A. (1977). A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337-343.
- Zhu, J., He, S., He, P., Liu, J. and Lyu, M. R. (2023). Loghub: a large collection of system log datasets for AI-driven log analytics. In *ISSRE 2023*.